React’s declarative model simplifies UI development, but as applications grow in size and complexity, performance bottlenecks can emerge—often due to unnecessary re-renders, inefficient data flows, or suboptimal component structures. For senior frontend engineers, diagnosing and addressing these issues is a critical skill. It is common in interviews to encounter questions such as:

- "How do you approach identifying and fixing performance issues in a React app?"
- "What tools would you use to analyze unnecessary re-renders, and how would you resolve them?"
- "Once you have identified components with frequent re-renders, how would you decide between using React Compiler, React.memo, splitting components, or restructuring state?"

To answer effectively, candidates should demonstrate familiarity with debugging strategies and performance optimization techniques such as:

- *React Developer Tools* (https://chromewebstore.google.com/detail/react-developer-tools/fmkadmapgofadopljbjfkapdkoienihi)—This browser extension enables inspection of the component tree, monitoring of re-renders, and real-time analysis of component performance. It provides valuable insights into props, state, context, and hooks.
- *Profiler API* (https://react.dev/reference/react/Profiler)—The Profiler API allows developers to measure the rendering performance of specific React components and identify bottlenecks that may lead to excessive or unnecessary re-renders.

To explore these tools in practice, let’s examine `RenderComponentExample` (listing 4.9, located in `use-cases/RenderComponentExample.tsx`). You can run it using: `VITE_NAME=RenderComponentExample npm run dev`.

Listing 4.9 RenderComponentExample

```
const ThemeContext = createContext<string>('black'); #A
const onRenderCallback: ProfilerOnRenderCallback = ( #B
  id: string,
  phase: 'mount' | 'update' | 'nested-update',
  actualDuration: number,
```